THESIS PROPOSAL OUTLINE

<THESIS TITLE>

<Sub-system (if / Module Name applicable)>

- **Student Name:** [Your Full Name]
- **Advisor 1:** [Name of Advisor 1] - **Staff ID:** [MSCB]
- **Advisor 2:** [Name of Advisor 2] - **Staff ID:** [MSCB]

I. INTRODUCTION

1. Problem Statement / Rationale

(Explain the context, the issues or challenges in current software engineering practices, and why this specific research topic is necessary and valuable.)

- ...
- ...

2. Research Objectives

- **General Objective:** *(The ultimate goal of your thesis.)*
- **Specific Objectives:** *(Measurable, step-by-step goals to achieve the general objective.)*
  - ...
  - ...

3. Scope and Delimitation

- **Research Subject (Target Object):** *(What software, system, or theoretical model are you studying/developing?)*
- **Research Scope:** *(The boundaries of your research: what is included, what is excluded, technologies used, and target environments.)*

II. THEORETICAL BACKGROUND / LITERATURE REVIEW

1. Core Concept 1 (e.g., Software Testing, Generative AI, etc.)

- **Definitions, Significance, and Standard Workflows:** ...
- **State-of-the-art Tools & Frameworks:** ...
- **Core References:**

  - Myers, G. J. (2011). *The Art of Software Testing*. Wiley.
  - IEEE Std 829-2008 – *Software and System Test Documentation*.

    - ….

- **2. Core Concept 2 / 3 (if applicable)**
- *(Add more theoretical sections depending on your thesis topic, e.g., Agentic Workflows, DevOps Pipeline integration, etc.)*

III. METHODOLOGY

1. Research Approach

(Describe your scientific approach, design methodologies, such as Agile/Scrum, Model-Driven Development, or empirical evaluation methods.)

...

2. Implementation Roadmap / Work Plan

No.	Tasks / Activities	Expected Deliverables
1	Literature Review & Tool Survey	Comprehensive Review Report
2	System Design	Architectural Models, Database Schema, Algorithm Flowcharts
3	Development & Implementation	Executable Codebase (Frontend, Backend, APIs)
4	Testing & Verification	Test Plan, Test Cases, Execution Log & Test Reports
5	Evaluation, Thesis Writing & Refinement	Final Evaluation Report & Completed Thesis Draft

IV. PRACTICAL APPLICATIONS AND DATASETS

1. Real-World Applications

(Describe how your research outcomes or prototypes can be deployed or used in practical software engineering pipelines.)

...

2. Experimental Datasets

Dataset	Description	Source/Link
Defects4J	A database of real, reproducible software bugs	https://github.com/rjust/defects4j
CodeSearchNet	Large-scale dataset for multilingual code representation	https://github.com/github/CodeSearchNet
OpenTelemetry	Observability framework for system behavior data	https://opentelemetry.io

V. RELATED WORK

(The following table serves as an illustrative example. Students should customize it based on their specific research focus.)

Project / Repository	Description	Reference Link
TestPilot (Microsoft)	An AI-driven assistant for automated unit test generation	https://github.com/githubnext/testpilot
AwesomeLLM4UT	A curated collection of research papers on LLM for Unit Testing	https://github.com/iSEngLab/AwesomeLLM4UT
LangChain + CI/CD Demo	Integrating LLM agents into continuous testing pipelines	https://github.com/langchain-ai/chain

VI. EXPECTED OUTCOMES AND CONTRIBUTIONS

(The following are examples. Customize these points to match your project deliverables.)

1. A functional prototype of an Autonomous AI Agent capable of automated test case generation and execution.
2. A lightweight testing pipeline was integrated successfully into small-to-medium-scale software projects.
3. A detailed performance evaluation report assessing the efficiency and limitations of the proposed approach, with future work suggestions.
4. *(Optional)* A systematic guide/reference material for subsequent research on AI-assisted Software Engineering.

VII. REFERENCES

*(**Crucial Note for Students:** You must manage your references using standard citation management tools such as **Mendeley** (Download Link) or **Zotero** (Download Link). Follow APA 7th style or IEEE format consistently.)*

1. Winteringham, M., 2024. *Software testing with generative AI*. Simon and Schuster.
2. Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing*. Wiley.
3. Naik, K., & Tripathy, P. (2011). *Software Testing: Principles and Practices*. Pearson Education.
4. Zhang et al. (2025). *Large Language Models for Unit Testing: A Systematic Literature Review*. arXiv:2503.10521.
5. Kohl et al. (2026). *Automated Structural Testing of LLM-based Agents*. arXiv:2601.18827.
6. Chu, B., Feng, Y., Liu, K., Guo, Z., Zhang, Y., Shi, H., ... & Xu, B. (2025). Large Language Models for Unit Test Generation: Achievements, Challenges, and Opportunities. *arXiv preprint arXiv:2511.21382*.
7. LangChain Documentation – https://python.langchain.com.
8. n8n Documentation – https://n8n.io.
9. Defects4J Dataset – https://github.com/rjust/defects4j.
10. CodeSearchNet Dataset – https://github.com/github/CodeSearchNet.